

ME228 Project Report

Single Image Super Resolution

Gaurav Kothari (24b2204)

Shaunik Jain (24b2183)

Vatsh Van (24b2259)

Vihaan Thaker (24b2130)

May 2, 2026

1 Introduction

Single Image Super Resolution (SISR) refers to the task of constructing a high resolution image from a single low resolution input. The main objective is to recover fine details and textures which might be lost due to factors like downsampling, noise, or compression. This is typically an illposed problem, as multiple high resolution images may be constructed from a single low resolution image, and hence the challenge lies in learning meaningful mappings to higher resolution images such that they are visually plausible. SISR has wide applications in fields such as medical imaging, satellite imagery, surveillance among others.

From Information Theory, we know that we cannot create new information inside the low resolution image which was not present from before, however, we can use Deep Learning techniques to learn this added difference from which a low resolution image becomes a high resolution image, and hence we use such models for the task of SISR. Such Deep Learning models are required to be trained on large datasets of corresponding low resolution and high resolution image pairs to learn complex mappings between the two domains. Typically for training, we use a high resolution (HR) image, and use a downsampled version of it as a low resolution (LR) input

An old technique used was SISR using Convolutional Neural Networks (CNNs), such models were called Super Resolution Convolutional Neural Networks (SRCNNs). They follow a simple yet effective convolutional architecture tailored for image super-resolution. It consists of a sequence of convolutional layers that operate directly on the input image using learnable filters (kernels). The first layer extracts relatively low-level features from the interpolated LR image, the intermediate layer maps these features non-linearly to a high-dimensional representation, and the final layer reconstructs the high-resolution image from these features. SRCNN however struggles to perform as good as the relatively newer models, a reason for that is that it has to reconstruct the whole HR image from ground 0, while newer architectures leverage residual connections so as to just learn the difference between the HR and the LR image, so learning how to map an LR image to a HR image becomes relatively simpler. Hence for the sake of this project, we have used a

2 Theory

We refer to this paper for the implementation of our Residual Network.

The model proposed takes as input an LR image of size $H \times W \times 3$, where H and W denote height and width, and 3 corresponds to the RGB channels. The first layer is a convolutional layer that transforms this input into a higher-dimensional representation by applying multiple learnable filters. Specifically, this layer maps the input to a tensor of size $H \times W \times 256$. Here, the number 256 refers to the number of *feature channels*, which can be understood as different learned responses (or feature maps) capturing various patterns such as edges, textures, and structures in the image.

After this initial transformation, the feature tensor is passed through a sequence of residual blocks. Each residual block consists of two convolutional layers, each preserving the spatial resolution ($H \times W$) and the number of feature channels (256). Thus, throughout all residual blocks, the feature tensor maintains a constant size of $H \times W \times 256$. A skip connection is used within each block, meaning that the input of the block is added element-wise to its output. This helps the network learn residual mappings and prevents degradation as depth increases. The final EDSR model uses **32** such residual blocks, so the feature tensor passes through **64** convolutional layers inside these blocks, all operating on tensors of identical size.

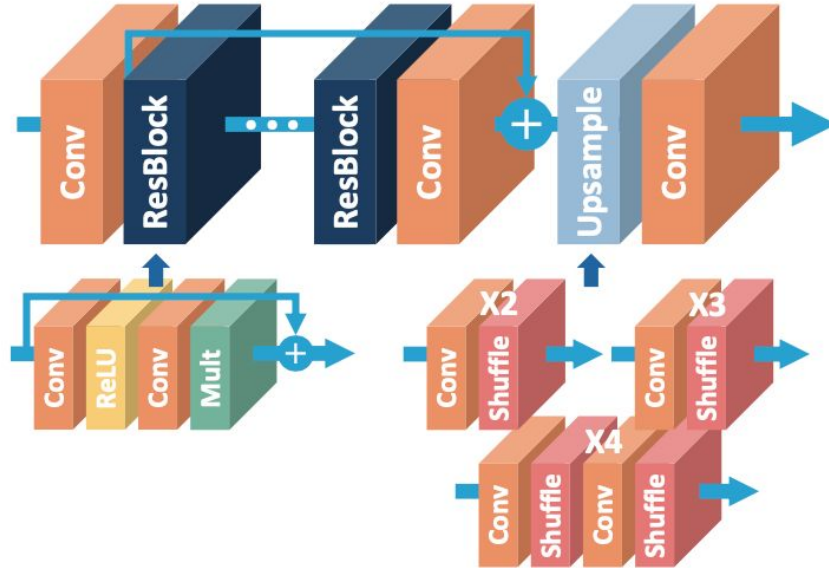


Figure 3: The architecture of the proposed single-scale SR network (EDSR).

After the residual blocks, another convolutional layer is applied, still maintaining the size $H \times W \times 256$. At this stage, all computations have been performed in the low-resolution space, which keeps the computational cost low because operations are done on

smaller spatial dimensions.

Upsampling is performed only at the end of the network. Suppose the desired scaling factor is s ($= 2$ for this project). Before upsampling, the network first increases the number of feature channels from 256 to $256 \times s^2$ using a convolutional layer. This results in a tensor of size $H \times W \times (256s^2)$. For example, for $s = 2$, this becomes $H \times W \times 1024$.

Next, a pixel shuffle (sub-pixel convolution) operation is applied. This operation rearranges the s^2 channels corresponding to each spatial location into an $s \times s$ neighborhood in the spatial domain. As a result, the spatial resolution increases from $H \times W$ to $(sH) \times (sW)$, while the number of channels reduces back to 256. Concretely, the tensor transforms from

$$H \times W \times (256s^2) \rightarrow (sH) \times (sW) \times 256.$$

Finally, a last convolutional layer maps this tensor to the output image of size $(sH) \times (sW) \times \mathbf{3}$, producing the high-resolution (HR) image.

3 Training Pipeline

For training, instead of feeding entire images into the network, the model uses smaller patches extracted from the low-resolution (LR) images. Specifically, random crops of size 48×48 are sampled from different spatial locations of the LR image. For each such LR patch, the corresponding high-resolution (HR) patch of size $(48s) \times (48s)$ is obtained from the ground-truth HR image at the same location. This random sampling is performed repeatedly during training, so that the model sees diverse regions (edges, textures, smooth areas) across different images. Using patches rather than full images reduces memory usage, allows larger batch sizes (here, 16), and increases the effective amount of training data through random cropping. Additionally, data augmentation is applied using random horizontal flips and 90° rotations to increase the diversity of training samples.

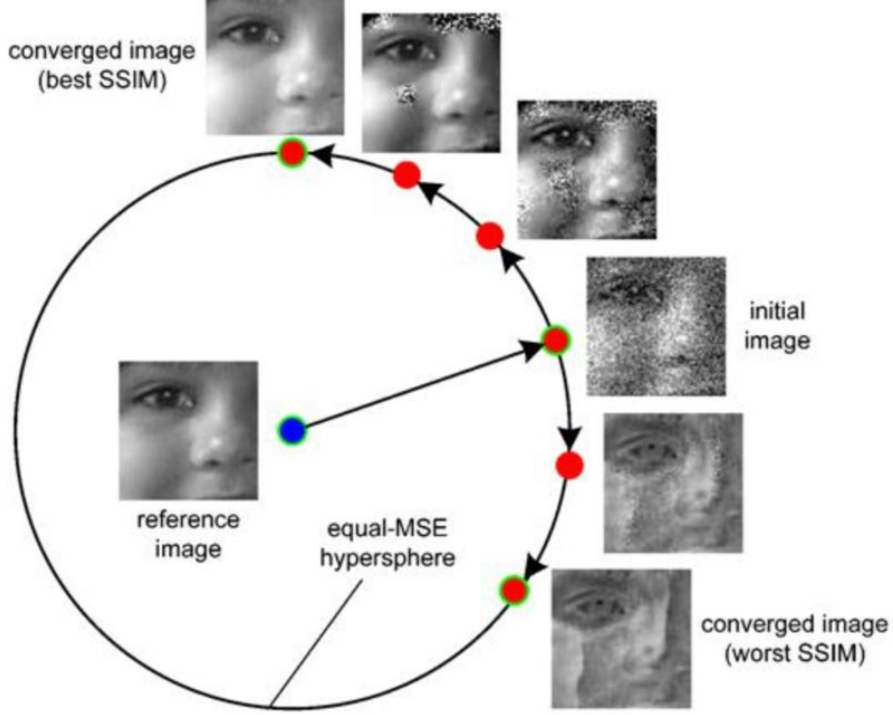
The model is trained using the L1 loss, which computes the average absolute difference between the predicted HR image and the ground-truth HR image. Compared to L2 loss (mean squared error), L1 loss is less sensitive to large errors and tends to produce sharper and more visually accurate reconstructions, whereas L2 often leads to overly smooth outputs. Empirically, L1 loss also provides more stable and faster convergence for super-resolution tasks.

During validation, the procedure differs from training. Instead of extracting random patches, the entire LR image is fed into the network to generate a full high-resolution output. This prediction is then compared with the corresponding ground-truth HR image using evaluation metrics such as PSNR and SSIM. No random cropping or augmentation is applied during validation, as the goal is to measure the true performance of the model on complete images.

We use the Div2k dataset for training our EDSR model, there are a total of 800 images from which we sample multiple 48×48 patches. All of the code that has been used by us for this project, has been uploaded on this GitHub Repository

We use evaluation metrics such as Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) to quantitatively compare the quality of reconstructed images against ground-truth high-resolution images.

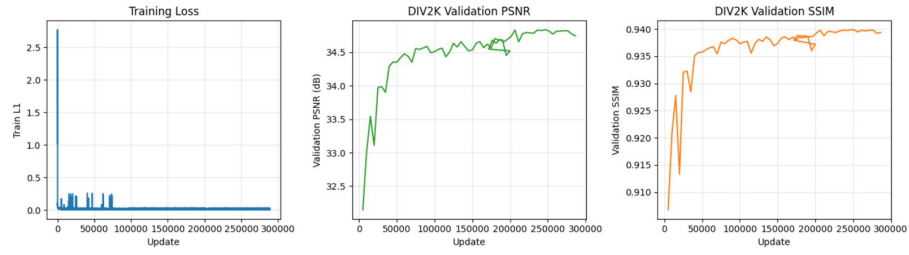
PSNR, measured in decibels (dB), is derived from the mean squared error and reflects the pixel-wise fidelity between the two images, with higher values indicating lower reconstruction error. However, PSNR does not account for perceptual quality or structural information.



SSIM, on the other hand, measures similarity based on luminance, contrast, and structural patterns, making it more aligned with human visual perception. As a result, SSIM is generally considered superior to PSNR for evaluating image quality, since it better captures the preservation of textures and structures rather than just pixel-level differences.

4 Results

We have trained the model for a total of 290k iterations, where one iteration refers to a single step of gradient descent. In each step of gradient descent, we randomly pick 16 (= batch size) patches of size 48 x 48 pixels randomly from the training set, and use it for this iteration for forward and backward pass, to train the whole network. The Validation PSNR and SSIM plots are plotted below.



And the PSNR, SSIM & L1 scores for Bicubic upsampling vs EDSR model are also plotted below

```

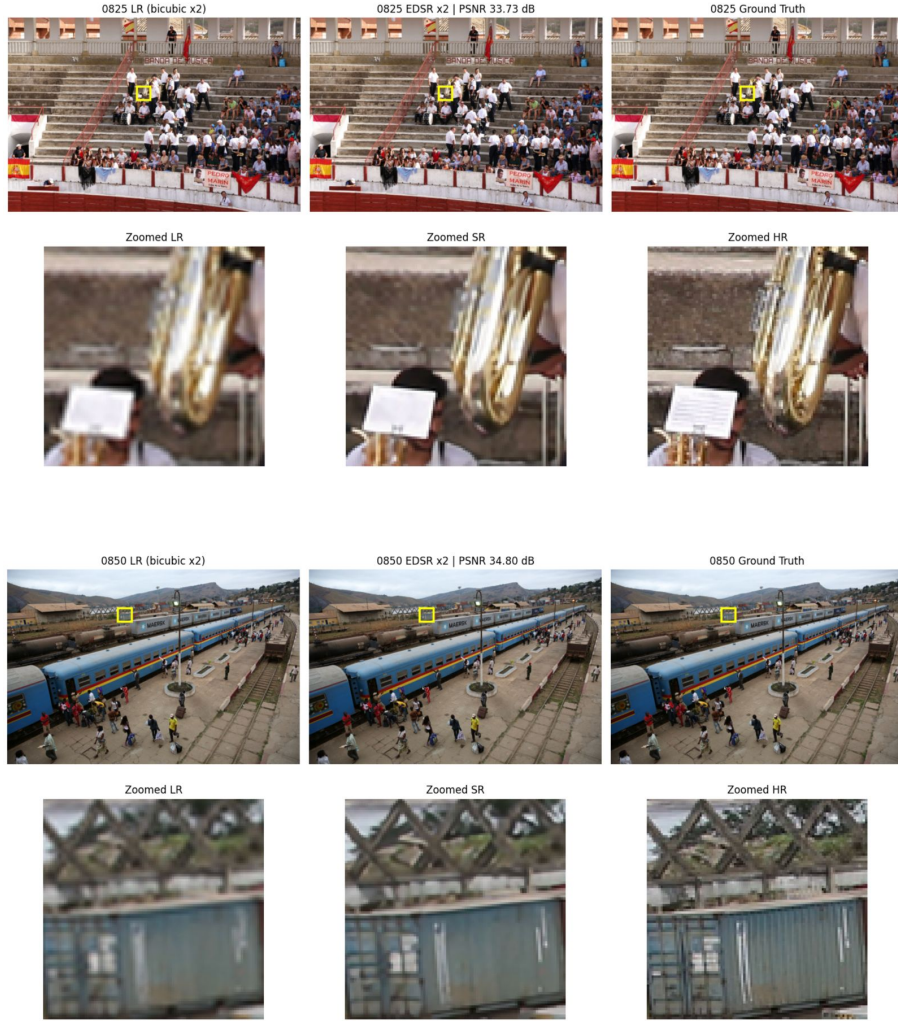
Loaded best checkpoint from update 235,000

=====
DIV2K VALIDATION RESULTS (RGB, shave = 6 + scale)
=====
Metric                Bicubic      EDSR x2
-----
PSNR (dB)             31.2523     34.8424
SSIM                   0.89808     0.93988
L1                     -           0.012348
=====

```

So our model clearly outperforms basic Bicubic upsampling. Further, below are the LR image, SR predicted image and the corresponding HR image that we have obtained from the model





5 Conclusion

Hence the model implemented successfully and meaningfully achieves the task of SISR ie improving the quality of images as what we can observe from the above results. However there are still a lot of points on which we can improve on (besides just increasing the complexity of the model ie increase the number of residual blocks, etc) listed below :-

1. We have only trained the model for around 290k iterations, while according to the paper, it is recommended to train it for around 1M iterations, however we could not achieve this due to time & compute constraints
2. In the training, we only use bicubically downsampled HR images as train images, so this makes our model a bit biased so as to only remove such sort of "bluriness", it would better, if we can add different types of noise to the HR data and then train the model on these different types of noise so as to be able to remove the "bluriness" in a better way

We would finally like to thank Prof. R Ganesh and Prof. Neeraj Kumbhakarna for giving us the chance to complete a project during this course.